

Table of Contents

The Long Running Process	3
Summary	3
Process Identity	4
Summary	4
Exit Codes and OS Feedback	5
Summary	5
Standard I/O Streams	6
Summary	6
Conditional Execution and Termination	7
Summary	7

Table of Figures

Figure 1: C code	3
Figure 2: Compiled the code	3
Figure 3: Running the code in background	3
Figure 4: Monitoring the process	3
Figure 5: C code in new file task2_identity.c	4
Figure 6: Terminal Execution and Verification	4
Figure 7: Code and termination in task3_exit.c	5
Figure 8: Code on task4_input	6
Figure 9: Terminal Execution	6
Figure 10: Code on task5_control.c	7
Figure 11: Testing Control Flow	7

The Long Running Process

```
GNU nano 8.7.1 task1_alive.c
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("I am starting...\n");
    for(int i = 1; i <= 30; i++) {
        sleep(1);
    }
    printf("I am finished.\n");
    return 0;
}
```

Figure 1: C code

```
kritika@kritika-VirtualBox:~$ gcc task1_alive.c -o task1
```

Figure 2: Compiled the code

```
kritika@kritika-VirtualBox:~/week3$ ./task1
$
I am starting...
I am finished.
```

Figure 3: Running the code in background

```
kritika@kritika-VirtualBox:~/week3$ ps aux | grep task1
kritika  7939  0.0  0.0 18000 2640 pts/2    S+   10:16
0:00 grep --color=auto task1
kritika@kritika-VirtualBox:~/week3$
```

Figure 4: Monitoring the process

Summary

This section shows how to create and run a program that stays active for a short period. The C code prints a start message, waits for 30 seconds using a loop with sleep commands, and then prints a finished message. It is compiled with GCC and checked in the terminal to see how the system handles running tasks.

Process Identity

```
GNU nano 8.7.1 task2_identity.c
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");
    sleep(20);
    return 0;
}
```

Figure 5: C code in new file task2_identity.c

```
kritika@kritika-VirtualBox:~/week3$ gcc task2_identity.c -o task2
kritika@kritika-VirtualBox:~/week3$ ./task2$
bash: ./task2$: No such file or directory
kritika@kritika-VirtualBox:~/week3$ ./task2 $
My PID is: 8121
My Parent PID is: 7715
Sleeping for 20 seconds...
```

Figure 6: Terminal Execution and Verification

Summary

This part explains how the system identifies programs running on your computer using unique numbers called PIDs and Parent PIDs. The C code asks the system for its own ID number and the ID number of the terminal that opened it. It displays these two numbers and then pauses for 20 seconds so you can verify the running task in the background.

Exit Codes and OS Feedback

```
#include <stdio.h>

int main() {
    int num; printf("Enter a num");
    scanf("%d", &num);
    if (num > 0) {
        printf("Success\n");
        return 0;
    }
    printf("Failure");
    return 1;
}
```

Terminal

```
kritika@kritika-VirtualBox:~/4th$ gcc task3_exit.c -o task3_exit
kritika@kritika-VirtualBox:~/4th$ ./task3_exit
Enter a num5
Success
kritika@kritika-VirtualBox:~/4th$
```

Figure 7: Code and termination in task3_exit.c

Summary

This section focuses on how programs tell the operating system whether they finished successfully or ran into an error. The program asks the user to enter a number. If the number is greater than zero, it prints a success message and returns a code of 0. If the number is zero or negative, it prints a failure message and returns a code of 1.

Standard I/O Streams

```
#include <stdio.h>

int main() {
    char name[50];

    printf("enter your name: ");
    scanf("%s", name);
    printf("Hello, %s! Welcome to OS Class.\n", name);
    return 0;
}
```

Figure 8: Code on task4_input

```
kritika@kritika-VirtualBox:~/4th$ touch task4_input.c
kritika@kritika-VirtualBox:~/4th$ nano task4_input.c
kritika@kritika-VirtualBox:~/4th$ gcc task4_input.c -o task4
kritika@kritika-VirtualBox:~/4th$ ./task4
enter your name: kriti
Hello, kriti! Welcome to OS Class.
```

Figure 9: Terminal Execution

Summary

This task shows basic communication between the user and the program through the terminal. The code asks the user to type in their name, stores what they typed into memory, and then prints out a personalized greeting welcoming them to the class.

Conditional Execution and Termination

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;

    printf("Current PID: %d\n", getpid());

    printf("Do you want to continue?");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0;
    } else {
        printf("Exiting");
        return 1;
    }
}
```

Figure 10: Code on task5_control.c

```
kritika@kritika-VirtualBox:~/4th$ nano task5_control.c
kritika@kritika-VirtualBox:~/4th$ nano task5_control.c
kritika@kritika-VirtualBox:~/4th$ gcc task5_control.c -o task5
kritika@kritika-VirtualBox:~/4th$ ./task5
Current PID: 10047
Do you want to continue?Y
Exitingkritika@kritika-VirtualBox:~/4th$ █
```

Figure 11: Testing Control Flow

Summary

This final section combines process IDs, user choices, and exit codes together. The code displays its process ID number and asks if you want to continue. If you type 1, it prints a message, pauses for 5 seconds, and finishes normally. If you type anything else, it immediately prints an exiting message and closes with an error status code.